

Doubly Linked List & Binary Search Tree

Table of Contents

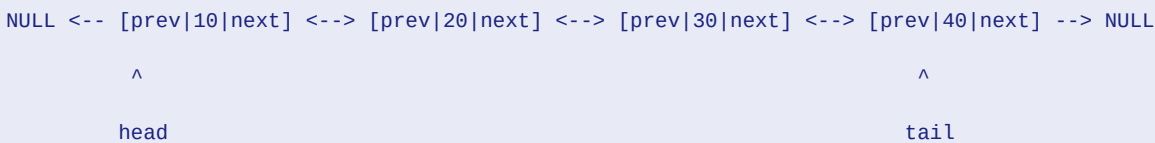
Section	Topic
1	Doubly Linked List — Structure & Node Definition
2	DLL — Insert at Start
3	DLL — Insert at End
4	DLL — Insert at Specific Position
5	DLL — Delete at Start
6	DLL — Delete at End
7	DLL — Delete at Specific Position
8	DLL — Complete C++ Program
9	Binary Search Tree — Concept & Properties
10	BST — Node Insertion (Algorithm + Code)
11	BST — Node Deletion (3 Cases + Algorithm + Code)
12	BST — Traversals (Inorder, Preorder, Postorder)
13	BST — Complete C++ Program
14	Complexity Summary & Cheat Sheet

Doubly Linked List

1. What is a Doubly Linked List?

A **Doubly Linked List (DLL)** is a linked list where each node has **three fields**: a **prev pointer** (to the previous node), a **data** field, and a **next pointer** (to the next node). Unlike a singly linked list, DLL allows **bidirectional traversal**. The prev of the head is NULL and the next of the tail is NULL.

Doubly Linked List — Visual Structure



Node Structure in C++

```
struct Node {
    int data;
    Node* prev; // pointer to previous node
    Node* next; // pointer to next node

    Node(int val) {
        data = val;
        prev = nullptr;
        next = nullptr;
    }
};
```

- **head**→**prev** is always **nullptr**.
- **tail**→**next** is always **nullptr**.
- Each node knows both its predecessor and successor — no traversal needed for many rewiring operations.

2. DLL — Insert at Start

Create a new node and prepend it before the current head. Update both prev/next links.

Step	Action
1	Create newNode with given value.
2	Set newNode → next = head .
3	Set newNode → prev = nullptr .
4	If list is non-empty: set head → prev = newNode .
5	Update head = newNode .

Insert at Start — Before & After

INSERT 5 at START

BEFORE: NULL <-- [10] <--> [20] <--> [30] --> NULL (head=10)

AFTER: NULL <-- [5] <--> [10] <--> [20] <--> [30] --> NULL (head=5)

```
void insertAtStart(Node*& head, int val) {
    Node* newNode = new Node(val);
    newNode->next = head;          // Step 2
    newNode->prev = nullptr;      // Step 3
    if (head != nullptr)         // Step 4
        head->prev = newNode;
    head = newNode;              // Step 5
}
```

■ Time Complexity: $O(1)$ — no traversal needed.

3. DLL — Insert at End

Traverse to the last node, then append the new node after it.

Step	Action
1	Create newNode with given value.
2	If list is empty: head = newNode. Stop.
3	Traverse to the tail (last node where curr→next == nullptr).
4	Set tail→next = newNode .
5	Set newNode→prev = tail .
6	newNode→next is already nullptr (from constructor).

Insert at End — Before & After

INSERT 50 at END

BEFORE: NULL <-- [10] <--> [20] <--> [30] --> NULL

AFTER: NULL <-- [10] <--> [20] <--> [30] <--> [50] --> NULL

```
void insertAtEnd(Node*& head, int val) {
    Node* newNode = new Node(val);
    if (head == nullptr) { head = newNode; return; }

    Node* curr = head;
    while (curr->next != nullptr) // reach tail
        curr = curr->next;

    curr->next = newNode; // Step 4
    newNode->prev = curr; // Step 5
}
```

■ Time Complexity: $O(n)$. Keeping a tail pointer makes this $O(1)$.

4. DLL — Insert at Specific Position

Position is **1-based**. Inserting at position 1 equals inserting at start. Otherwise traverse to position (pos-1), then wire the new node between that node and its successor.

Step	Action
1	If pos == 1: call insertAtStart and return.
2	Traverse to node at position pos-1 ; call it prevNode .
3	If prevNode is nullptr: position is out of range — print error.
4	Let nextNode = prevNode→next .
5	Set newNode→prev = prevNode and newNode→next = nextNode .
6	Set prevNode→next = newNode .
7	If nextNode is not nullptr: set nextNode→prev = newNode .

Insert at Position 3 — Before & After

INSERT 25 at position 3 (between 20 and 30)

BEFORE: NULL <-- [10] <--> [20] <--> [30] <--> [40] --> NULL

^

pos-1=2

AFTER: NULL <-- [10] <--> [20] <--> [25] <--> [30] <--> [40] --> NULL

```
void insertAtPosition(Node*& head, int val, int pos) {
    if (pos == 1) { insertAtStart(head, val); return; }

    Node* newNode = new Node(val);
    Node* prevNode = head;

    for (int i = 1; i < pos - 1; i++) { // reach node at pos-1
        if (prevNode == nullptr) break;
        prevNode = prevNode->next;
    }

    if (prevNode == nullptr) {
        cout << "Position out of range!" << endl;
        delete newNode;
        return;
    }

    Node* nextNode = prevNode->next; // Step 4
    newNode->prev = prevNode;        // Step 5a
    newNode->next = nextNode;        // Step 5b
    prevNode->next = newNode;        // Step 6
    if (nextNode != nullptr)        // Step 7
        nextNode->prev = newNode;
}
```

■ Time Complexity: $O(n)$ in worst case (traversal to position).

5. DLL — Delete at Start

Remove the head node and make its successor the new head.

Step	Action
1	Edge case: if <code>head == nullptr</code> , print error and return.
2	Save <code>temp = head</code> .
3	Move <code>head = head→next</code> .
4	If new head is not <code>nullptr</code> : set <code>head→prev = nullptr</code> .
5	Delete <code>temp</code> .

Delete at Start — Before & After

DELETE at START (remove 10)

BEFORE: NULL <-- [10] <--> [20] <--> [30] --> NULL (head=10)

AFTER: NULL <-- [20] <--> [30] --> NULL (head=20)

```
void deleteAtStart(Node*& head) {
    if (head == nullptr) { cout << "Empty list!\n"; return; }

    Node* temp = head;          // Step 2
    head = head->next;          // Step 3
    if (head != nullptr)        // Step 4
        head->prev = nullptr;
    delete temp;                // Step 5
}
```

■ Time Complexity: $O(1)$.

6. DLL — Delete at End

Traverse to the tail, then make the second-to-last node the new tail.

Step	Action
1	Edge case: empty list.
2	Edge case: single node — delete it, set head = nullptr.
3	Traverse to the tail (<code>curr->next == nullptr</code>).
4	Set tail → prev → next = nullptr (second-to-last becomes new tail).
5	Delete old tail.

Delete at End — Before & After

DELETE at END (remove 40)

BEFORE: NULL <-- [10] <--> [20] <--> [30] <--> [40] --> NULL

AFTER: NULL <-- [10] <--> [20] <--> [30] --> NULL

```
void deleteAtEnd(Node*& head) {
    if (head == nullptr) { cout << "Empty!\n"; return; }
    if (head->next == nullptr) { delete head; head = nullptr; return; }

    Node* curr = head;
    while (curr->next != nullptr)    // reach tail
        curr = curr->next;

    curr->prev->next = nullptr;      // Step 4
    delete curr;                   // Step 5
}
```

■ Time Complexity: $O(n)$. $O(1)$ if a tail pointer is maintained.

7. DLL — Delete at Specific Position

Traverse to the node at the given position. Rewire its predecessor's next and successor's prev to bypass it, then delete it.

Step	Action
1	If <code>pos == 1</code> : call <code>deleteAtStart</code> and return.
2	Traverse to the node at pos ; call it curr .
3	If <code>curr</code> is <code>nullptr</code> : position out of range.
4	If <code>curr</code> → prev is not <code>nullptr</code> : set <code>curr</code> → prev → next = <code>curr</code> → next .
5	If <code>curr</code> → next is not <code>nullptr</code> : set <code>curr</code> → next → prev = <code>curr</code> → prev .
6	Delete curr .

Delete at Position 3 — Before & After

DELETE node at position 3 (remove 30)

BEFORE: NULL <-- [10] <--> [20] <--> [30] <--> [40] --> NULL

^

pos=3

AFTER: NULL <-- [10] <--> [20] <--> [40] --> NULL

```
void deleteAtPosition(Node*& head, int pos) {
    if (head == nullptr) { cout << "Empty!\n"; return; }
    if (pos == 1) { deleteAtStart(head); return; }

    Node* curr = head;
    for (int i = 1; i < pos; i++) {
        if (curr == nullptr) break;
        curr = curr->next;
    }

    if (curr == nullptr) { cout << "Position out of range!\n"; return; }

    if (curr->prev != nullptr) // Step 4
        curr->prev->next = curr->next;
    if (curr->next != nullptr) // Step 5
        curr->next->prev = curr->prev;

    delete curr; // Step 6
}
```

■ **Time Complexity:** $O(n)$ traversal. The rewiring itself is $O(1)$ — a DLL advantage over SLL.

8. DLL — Complete C++ Program

```
#include <iostream>
using namespace std;

struct Node {
    int data; Node* prev; Node* next;
    Node(int v) : data(v), prev(nullptr), next(nullptr) {}
};

void insertAtStart(Node*& head, int val) {
    Node* n = new Node(val);
    n->next = head;
    if (head) head->prev = n;
    head = n;
}

void insertAtEnd(Node*& head, int val) {
    Node* n = new Node(val);
    if (!head) { head = n; return; }
    Node* c = head;
    while (c->next) c = c->next;
    c->next = n; n->prev = c;
}

void insertAtPosition(Node*& head, int val, int pos) {
    if (pos == 1) { insertAtStart(head, val); return; }
    Node* n = new Node(val);
    Node* p = head;
    for (int i = 1; i < pos-1 && p; i++) p = p->next;
    if (!p) { cout << "Out of range\n"; delete n; return; }
    Node* nx = p->next;
    n->prev = p; n->next = nx;
    p->next = n;
    if (nx) nx->prev = n;
}

void deleteAtStart(Node*& head) {
    if (!head) return;
    Node* t = head; head = head->next;
    if (head) head->prev = nullptr;
    delete t;
}

void deleteAtEnd(Node*& head) {
    if (!head) return;
    if (!head->next) { delete head; head = nullptr; return; }
    Node* c = head;
    while (c->next) c = c->next;
    c->prev->next = nullptr; delete c;
}

void deleteAtPosition(Node*& head, int pos) {
    if (!head) return;
    if (pos == 1) { deleteAtStart(head); return; }
    Node* c = head;
    for (int i = 1; i < pos && c; i++) c = c->next;
    if (!c) { cout << "Out of range\n"; return; }
    if (c->prev) c->prev->next = c->next;
```



```

        if (c->next) c->next->prev = c->prev;
        delete c;
    }

void display(Node* head) {
    while (head) {
        cout << head->data;
        if (head->next) cout << " <--> ";
        head = head->next;
    }
    cout << endl;
}

int main() {
    Node* head = nullptr;
    insertAtEnd(head, 10);
    insertAtEnd(head, 20);
    insertAtEnd(head, 30);
    insertAtEnd(head, 40);
    insertAtStart(head, 5);
    insertAtPosition(head, 25, 4); // insert 25 at pos 4
    cout << "List: "; display(head);

    deleteAtStart(head);
    deleteAtEnd(head);
    deleteAtPosition(head, 2);
    cout << "After deletions: "; display(head);
    return 0;
}

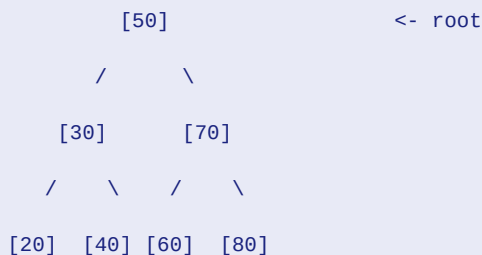
```

Binary Search Tree

9. What is a Binary Search Tree?

A **Binary Search Tree (BST)** is a binary tree where each node follows the **BST Property**: all values in the **left subtree** are **less than** the node's value, and all values in the **right subtree** are **greater than** the node's value. This ordering makes search, insertion, and deletion efficient.

Binary Search Tree — Example



BST Property: $\text{left} < \text{parent} < \text{right}$ (holds at EVERY node)

- Left subtree values are **strictly less than** the parent.
- Right subtree values are **strictly greater than** the parent.
- **No duplicate values** in a standard BST.
- Average height is $O(\log n)$; worst case (skewed) is $O(n)$.
- **In-order traversal** of a BST always gives a **sorted sequence**.

BST Node Structure in C++

```
struct Node {
    int data;
    Node* left; // left child (smaller values)
    Node* right; // right child (larger values)

    Node(int val) {
        data = val;
        left = nullptr;
        right = nullptr;
    }
};
```

10. BST — Node Insertion

Insertion in a BST is done by comparing the new value with the current node: go **left** if smaller, go **right** if larger, until a `nullptr` (empty slot) is found. This naturally preserves the BST property.

Algorithm — BST Insert (Recursive)

Step	Action
------	--------

1	If root is nullptr: create a new node and return it as the new root.
2	If $val < root \rightarrow data$: recursively insert into left subtree ; $root \rightarrow left = result$.
3	If $val > root \rightarrow data$: recursively insert into right subtree ; $root \rightarrow right = result$.
4	If $val == root \rightarrow data$: duplicate — ignore (or handle per requirements).
5	Return root (unchanged for non-empty nodes).

BST Insertion Trace — Inserting 45

```

INSERT 45 into BST:      [50]
                        /  \
45 < 50 -> go left      [30] [70]
45 > 30 -> go right     /  \
45 > 40 -> go right    [20] [40]
40->right is NULL      \
Insert here!           [45] <-- new node

```

```

// Recursive insert – returns updated root
Node* insert(Node* root, int val) {
    // Base case: empty spot found
    if (root == nullptr)
        return new Node(val);

    if (val < root->data)
        root->left = insert(root->left, val); // go left
    else if (val > root->data)
        root->right = insert(root->right, val); // go right
    // val == root->data: duplicate, do nothing

    return root;
}

// Usage:
// Node* root = nullptr;
// root = insert(root, 50);

```

■ **Time Complexity:** $O(h)$ where h = height of tree. Average $O(\log n)$, worst $O(n)$ for skewed tree.

11. BST — Node Deletion

Deletion is the most complex BST operation because we must preserve the BST property. There are **three cases** depending on the number of children the target node has.

Case 1: Node has NO children (Leaf Node)

Simply delete the node. Set its parent's pointer to nullptr.

Case 2: Node has ONE child

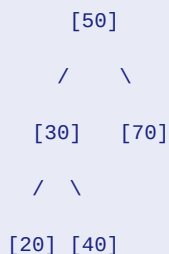
Bypass the node: link its parent directly to its single child. Then delete the node.

Case 3: Node has TWO children

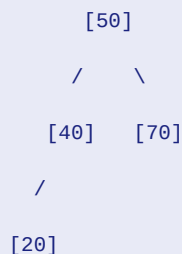
Find the In-order Successor (smallest node in right subtree). Copy its value into the target node. Then delete the in-order successor (which has at most one child — recurse).

BST Deletion — Case 3 Example

ORIGINAL BST:



DELETE 30 (two children):



In-order successor of 30 = 40 (smallest in right subtree of 30)

Copy 40 into node 30, then delete original node 40 (leaf)

Algorithm — BST Delete (Recursive)

Step	Action
1	If root is nullptr: node not found, return nullptr.
2	If val < root→data: delete from left subtree ; root→left = result.
3	If val > root→data: delete from right subtree ; root→right = result.
4	If val == root→data: this is the node to delete — apply one of the 3 cases.
4a	Case 1 (no children): delete node, return nullptr.
4b	Case 2 (one child): save child, delete node, return child.
4c	Case 3 (two children): find in-order successor (min of right subtree). Copy its value here. Delete in-order successor from right subtree.
5	Return updated root.

```
// Helper: find minimum node in a subtree
Node* findMin(Node* root) {
    while (root->left != nullptr)
```

```

        root = root->left;
    return root;
}

// Recursive BST deletion
Node* deleteNode(Node* root, int val) {
    if (root == nullptr) return nullptr; // Step 1: not found

    if (val < root->data) // Step 2: go left
        root->left = deleteNode(root->left, val);
    else if (val > root->data) // Step 3: go right
        root->right = deleteNode(root->right, val);
    else { // Step 4: found!
        // Case 1: Leaf node
        if (!root->left && !root->right) {
            delete root;
            return nullptr;
        }
        // Case 2a: Only right child
        if (!root->left) {
            Node* temp = root->right;
            delete root;
            return temp;
        }
        // Case 2b: Only left child
        if (!root->right) {
            Node* temp = root->left;
            delete root;
            return temp;
        }
        // Case 3: Two children
        Node* successor = findMin(root->right); // in-order successor
        root->data = successor->data; // copy value
        root->right = deleteNode(root->right, successor->data); // delete successor
    }
    return root;
}

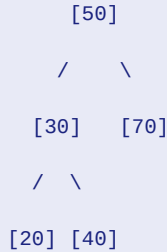
```

■ Time Complexity: $O(h)$. Finding in-order successor also takes $O(h)$. Overall $O(\log n)$ average, $O(n)$ worst.

12. BST — Tree Traversals

BST supports three classic DFS traversals. Each visits every node exactly once in a specific order. For the tree below, expected outputs are shown.

Example BST for Traversal Outputs



Traversal	Visit Order	Output	Use Case
In-order	Left → Root → Right	20, 30, 40, 50, 70	Sorted output from BST
Pre-order	Root → Left → Right	50, 30, 20, 40, 70	Copy/serialize the tree
Post-order	Left → Right → Root	20, 40, 30, 70, 50	Delete the tree safely

```
void inorder(Node* root) {    // Left-Root-Right (sorted)
    if (!root) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}

void preorder(Node* root) {   // Root-Left-Right
    if (!root) return;
    cout << root->data << " ";
    preorder(root->left);
    preorder(root->right);
}

void postorder(Node* root) {  // Left-Right-Root
    if (!root) return;
    postorder(root->left);
    postorder(root->right);
    cout << root->data << " ";
}
```

13. BST — Complete C++ Program

```
#include <iostream>
using namespace std;

struct Node {
    int data; Node* left; Node* right;
    Node(int v) : data(v), left(nullptr), right(nullptr) {}
};

Node* insert(Node* root, int val) {
    if (!root) return new Node(val);
```

```

    if (val < root->data) root->left = insert(root->left, val);
    else if (val > root->data) root->right = insert(root->right, val);
    return root;
}

Node* findMin(Node* root) {
    while (root->left) root = root->left;
    return root;
}

Node* deleteNode(Node* root, int val) {
    if (!root) return nullptr;
    if (val < root->data) root->left = deleteNode(root->left, val);
    else if (val > root->data) root->right = deleteNode(root->right, val);
    else {
        if (!root->left && !root->right) { delete root; return nullptr; }
        if (!root->left) { Node* t=root->right; delete root; return t; }
        if (!root->right) { Node* t=root->left; delete root; return t; }
        Node* succ = findMin(root->right);
        root->data = succ->data;
        root->right = deleteNode(root->right, succ->data);
    }
    return root;
}

bool search(Node* root, int val) {
    if (!root) return false;
    if (val == root->data) return true;
    if (val < root->data) return search(root->left, val);
    return search(root->right, val);
}

void inorder(Node* root) { if(!root)return; inorder(root->left); cout<<root->data<<" "; inorder(root->right); }
void preorder(Node* root) { if(!root)return; cout<<root->data<<" "; preorder(root->left); preorder(root->right); }
void postorder(Node* root) { if(!root)return; postorder(root->left); postorder(root->right); cout<<root->data<<" "; }

int main() {
    Node* root = nullptr;
    int vals[] = {50, 30, 70, 20, 40, 60, 80};
    for (int v : vals) root = insert(root, v);

    cout << "Inorder: "; inorder(root); cout << endl;
    cout << "Preorder: "; preorder(root); cout << endl;
    cout << "Postorder: "; postorder(root); cout << endl;

    root = deleteNode(root, 30); // delete node with two children
    cout << "After delete 30: "; inorder(root); cout << endl;

    cout << "Search 40: " << (search(root,40) ? "Found" : "Not found") << endl;
    return 0;
}

```

14. Complexity Summary

Operation	Structure	Avg Time	Worst Time	Space	Notes
Insert at Start	DLL	$O(1)$	$O(1)$	$O(1)$	No traversal needed
Insert at End	DLL	$O(n)$	$O(n)$	$O(1)$	$O(1)$ with tail pointer
Insert at Pos	DLL	$O(n)$	$O(n)$	$O(1)$	Traverse to pos-1
Delete at Start	DLL	$O(1)$	$O(1)$	$O(1)$	No traversal needed
Delete at End	DLL	$O(n)$	$O(n)$	$O(1)$	$O(1)$ with tail pointer
Delete at Pos	DLL	$O(n)$	$O(n)$	$O(1)$	Traverse to pos
Insert	BST	$O(\log n)$	$O(n)$	$O(h)$	h =height; $O(n)$ skewed
Delete	BST	$O(\log n)$	$O(n)$	$O(h)$	3 cases; $O(n)$ skewed
Search	BST	$O(\log n)$	$O(n)$	$O(h)$	Binary comparison
Traversal	BST	$O(n)$	$O(n)$	$O(h)$	Visits all nodes once

Quick-Reference Cheat Sheet

Operation	Key Logic / Pointer Updates
DLL Insert Start	<code>newNode→next=head; if(head) head→prev=newNode; head=newNode</code>
DLL Insert End	<code>traverse to tail; tail→next=newNode; newNode→prev=tail</code>
DLL Insert Pos	<code>traverse to pos-1; wire: prev→next=new, new→prev=prev, new→next=next, next→prev=new</code>
DLL Delete Start	<code>temp=head; head=head→next; if(head) head→prev=nullptr; delete temp</code>
DLL Delete End	<code>traverse to tail; tail→prev→next=nullptr; delete tail</code>
DLL Delete Pos	<code>traverse to pos; prev→next=curr→next; next→prev=curr→prev; delete curr</code>
BST Insert	<code>if(valroot) go right; insert at nullptr</code>
BST Delete Leaf	<code>delete node; return nullptr to parent</code>
BST Delete 1 child	<code>return the single child to parent; delete node</code>
BST Delete 2 child	<code>copy in-order successor value; delete successor from right subtree</code>
In-order	<code>Left → Root → Right (gives sorted sequence)</code>
Pre-order	<code>Root → Left → Right (useful for tree copy)</code>
Post-order	<code>Left → Right → Root (useful for tree deletion)</code>